Advanced Tree Techniques

Link-cut trees, DSU on tree, virtual trees, rerooting DP

Learning goals

- Maintain a **dynamic forest** (link, cut, reroot, path aggregates) in O(log n) amortised with link-cut trees
- Answer all-subtree queries in O(n log n) with **DSU on tree** ("sack")
- Compress a tree to the k relevant vertices with a **virtual tree** in O(k log n)
- Compute a DP for **every root** at once with rerooting, including non-invertible merges
- Choose between these and HLD / centroid (*nationals-04*)

Which tool?

Problem shape	Technique
edges added *and removed*, path queries, online	**link-cut tree**
static tree, a query per subtree, "count distinct / most frequent"	**DSU on tree**
many queries, each on a small vertex set, $\Sigma k \le 10^5$	**virtual tree**
answer for every vertex as the root	**rerooting**
static tree, path updates/queries	HLD (*nationals-04*)
counting paths with a property	centroid decomposition (*nationals-04*)

Link-cut trees: the model

- Represented forest: rooted trees that change by **link** (add edge) and **cut**
- Each tree is split into **preferred paths** (vertex-disjoint, root-to-leaf direction)
- Each preferred path is stored in a **splay tree keyed by depth**
- The splay root's parent pointer = **path-parent** (a "virtual" edge to the vertex above the path)

```
actual tree          preferred paths        splay trees
    1                  1–2–4  (path A)      A: in-order 1,2,4
   / \                 3      (path B)      B: {3}, path-parent → 1
  2   3                5      (path C)      C: {5}, path-parent → 2
 / \
4   5
```

access(v): the only real operation

Make root → v a single preferred path, with v deepest on it.

```
access(v):
    splay(v); cut off v's right child (deeper part)  → becomes its own path
    while v's splay has a path-parent w:
        splay(w); replace w's right child by v's splay tree
        splay(v)
```

After access(v): v is the splay root, its splay holds exactly root...v.

- isRoot(x): x is not a child of its parent (parent pointer is a path-parent)

Everything else from access

Operation	Implementation
makeRoot(v)	access(v), then **reverse** v's splay (lazy flag)
findRoot(v)	access(v), walk to leftmost node, splay it
link(u, v)	makeRoot(u); set parent(u) = v (if different trees)
cut(u, v)	makeRoot(u); access(v); now u must be v's left child with no right child; detach
path(u, v)	makeRoot(u); access(v); answer = aggregate at v
LCA(u, v) (fixed root)	access(u); the last path-parent jump in access(v)

LCT code: splay part

```
bool isRoot(int x) { int p = t[x].p;
    return p == 0 || (t[p].ch[0] != x && t[p].ch[1] != x); }
void pull(int x) { t[x].sum = t[t[x].ch[0]].sum + t[x].val + t[t[x].ch[1]].sum; }
void flip(int x) { if (x) { swap(t[x].ch[0], t[x].ch[1]); t[x].rev ^= 1; } }
void push(int x) { if (t[x].rev) { flip(t[x].ch[0]); flip(t[x].ch[1]); t[x].rev = false; } }
void rotate(int x) {
    int p = t[x].p, g = t[p].p, d = t[p].ch[1] == x;
    if (!isRoot(p)) t[g].ch[t[g].ch[1] == p] = x;   // test BEFORE relinking
    t[x].p = g;
    t[p].ch[d] = t[x].ch[d ^ 1]; if (t[x].ch[d ^ 1]) t[t[x].ch[d ^ 1]].p = p;
    t[x].ch[d ^ 1] = p; t[p].p = x;
    pull(p); pull(x);
}
void splay(int x) {
    static vector<int> stk; stk.clear();
    for (int y = x; ; y = t[y].p) { stk.push_back(y); if (isRoot(y)) break; }
    while (!stk.empty()) push(stk.back()), stk.pop_back();   // lazies top-down
    while (!isRoot(x)) {
        int p = t[x].p, g = t[p].p;
        if (!isRoot(p)) rotate((t[g].ch[1] == p) == (t[p].ch[1] == x) ? p : x);
        rotate(x);
    }
}
```

LCT code: tree operations

```
void access(int x) {
    for (int last = 0, y = x; y; last = y, y = t[y].p) {
        splay(y); t[y].ch[1] = last; pull(y);
    }
    splay(x);
}
void makeRoot(int x) { access(x); flip(x); }
int findRoot(int x) {
    access(x);
    while (t[x].ch[0]) push(x), x = t[x].ch[0];
    splay(x); return x;
}
bool link(int u, int v) {
    makeRoot(u); if (findRoot(v) == u) return false;
    t[u].p = v; return true;
}
bool cut(int u, int v) {
    makeRoot(u); access(v);
    if (t[v].ch[0] != u || t[u].ch[1] != 0) return false;
    t[v].ch[0] = t[u].p = 0; pull(v); return true;
}
ll pathSum(int u, int v) { makeRoot(u); access(v); return t[v].sum; }
```

Why O(log n) amortised

- **Splay cost**: access performs splays whose total cost is O(log n) amortised + O(#preferred-child changes)
- **Preferred-child changes**: heavy-light argument
  - a change that makes a *light* edge preferred: ≤ log n per access
  - a change that makes a *heavy* edge preferred is paid for by an earlier heavy → non-preferred change
  - total O(m log n) over m operations
- Combined with the splay potential: **O(log n) amortised per operation**
- Constant is large (~3–5× a segment tree); $2 \cdot 10^5$ ops in well under a second

Edge weights and other aggregates

- LCT stores values on **vertices**. For edge weights: create a node per edge, link u–e–v
- Path max with position → dynamic **MST**: add edge (u, v, w); if connected, find max edge on path, cut it if heavier
- Any monoid works: sum, max, min, (matrix) products — mind **reversal** for non-commutative ones (store both directions)
- Subtree aggregates: maintain "virtual subtree" sums of non-preferred children (update in access)

LCT pitfalls

- `rotate`: test `isRoot(p)` **before** changing pointers
- `splay`: push lazy flags from the splay root **down** to x first
- `findRoot`: push while descending; splay the found root (else amortisation breaks)
- `cut`: verify the edge exists (u directly left of v with no right child)
- Mixing makeRoot with a "fixed root" interpretation → wrong LCA
- Tested: 300 random runs × 400 ops vs naive BFS forest (link, cut, connected, path sum, point set)

DSU on tree (sack)

Codeforces 600E: each vertex has a colour; for **every** subtree, sum of the colours that occur most often. $n \le 10^5$.

- Naive: recompute counts per subtree → $O(n^2)$
- Idea: keep the count array of the **heavy** child, re-add light subtrees

```
dfs(v, keep):
    for light child c: dfs(c, keep = false)    // computes answers, then wipes
    dfs(heavy child, keep = true)              // its counts stay in the array
    add all vertices of light subtrees, add v
    answer queries for v
    if !keep: remove all of subtree(v)
```

Why O(n log n)

- A vertex u is (re)added once for every **light edge** on the path root → u
- Any root-to-vertex path has ≤ $\log_2 n$ light edges (subtree size at least doubles crossing one)
- Removal mirrors addition
- Total **O(n log n)** add/remove operations; each is O(1) here (or O(log) with a BIT)

Iterate light subtrees via the **Euler tour** range [tin, tout]: no recursion for adding, cache-friendly.

DSU on tree: code

```
void dfs(int v, int p, bool keep) {
    for (int c : g[v]) if (c != p && c != heavy[v]) dfs(c, v, false);
    if (heavy[v]) dfs(heavy[v], v, true);
    for (int c : g[v]) if (c != p && c != heavy[v])
        for (int t = tin[c]; t <= tout[c]; t++) add(ver[t]);
    add(v);
    ans[v] = sumMax;
    if (!keep) {
        for (int t = tin[v]; t <= tout[v]; t++) cnt[col[ver[t]]]--;
        maxCnt = 0; sumMax = 0;                 // everything is gone: reset globals
    }
}
void add(int v) {
    int c = col[v];
    if (++cnt[c] > maxCnt) maxCnt = cnt[c], sumMax = c;
    else if (cnt[c] == maxCnt) sumMax += c;
}
```

"max" is not removable, but we only ever remove **everything**, so resetting is valid.

Sack vs small-to-large vs Mo on tree

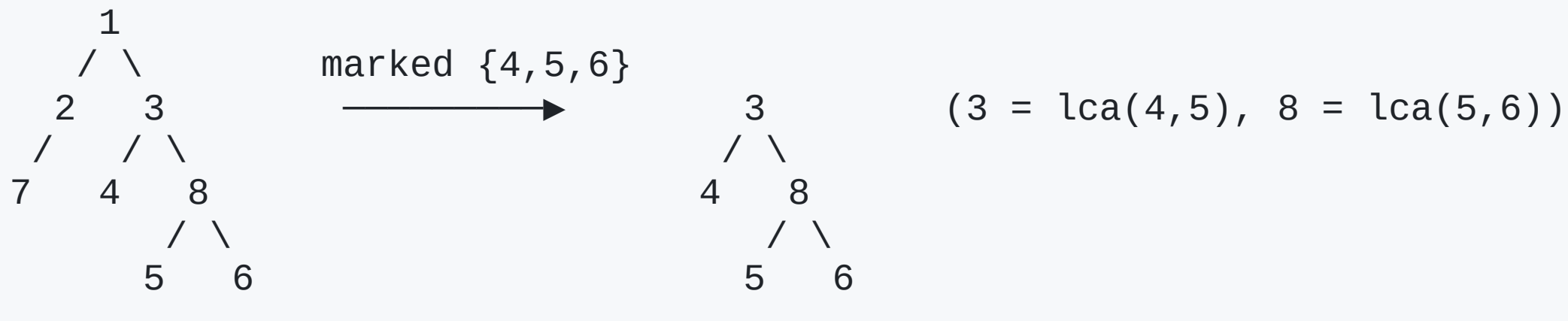
	DSU on tree	small-to-large (*nationals-04*)	Mo on Euler tour
Time	O(n log n)	O(n log² n) with sets	O(n√n)
Memory	one global array	one container per vertex	one global array
Needs removal?	only "remove all"	no	yes
Online?	no	no	no

Use sack when the per-subtree statistic lives in a global array and supports add + wipe.

Virtual (auxiliary) trees

Queries: "given k marked vertices, compute something", $\Sigma k \le 2 \cdot 10^5$, $n \le 2 \cdot 10^5$.

- An O(n) DP per query → O(nq): too slow
- Only marked vertices and **branching points** matter
- Virtual tree = marked vertices ∪ pairwise LCAs, edges = compressed paths
- Size ≤ 2k − 1, built in **O(k log n)**



Building it

Lemma. Sort marked vertices by DFS entry time: $a_1, \ldots, a_k$. Then $\{a_i\} \cup \{\mathrm{lca}(a_i, a_{i+1})\}$ is closed under LCA.

Why: for any two marked vertices, their LCA equals the LCA of some **adjacent** pair in DFS order (the shallowest of the adjacent LCAs between them).

```
sort(vs.begin(), vs.end(), byTin);
int k = vs.size();
for (int i = 0; i + 1 < k; i++) vs.push_back(lca(vs[i], vs[i + 1]));
sort(vs.begin(), vs.end(), byTin);
vs.erase(unique(vs.begin(), vs.end()), vs.end());
for (size_t i = 1; i < vs.size(); i++) vpar[vs[i]] = lca(vs[i - 1], vs[i]);
```

Parent of each vertex = LCA with its predecessor in DFS order. Process in **reverse** order for bottom-up DP.

Example: cut off all marked vertices

[SDOI2011] 消耗战 (Luogu P2495): delete edges of minimum total weight so that no marked vertex is connected to root 1.

- `mn[v]` = minimum edge weight on root → v (precomputed once)
- On the virtual tree, bottom-up:

```
cost(v) = mn[v]                          if v is marked
cost(v) = min(mn[v], Σ cost(children)) otherwise
answer = Σ cost(children of 1)
```

- Per query O(k log n); reset only touched vertices
- Codeforces 613D *Kingdom and its Cities* is the same pattern with a different DP

Rerooting DP

Compute f(r) = "DP of the whole tree rooted at r" for **every** r.

1. Root at 1, compute `down[v]` from children (post-order)
2. Compute `up[v]` = contribution of everything **outside** subtree(v), seen from v (pre-order)
3. Combine: f(v) = merge(down[v], up[v])

Sum of distances (CSES 1133): when the root moves p → v, sz[v] vertices get closer, n − sz[v] get farther:

```
all[v] = all[p] − sz[v] + (n − sz[v])
```

Rerooting: non-invertible merges

When merge is max/min (can't "subtract" a child), use **prefix/suffix** over the children of p:

```
// up[c] = 1 + best path from p that avoids child c
for (int i = 0; i < k; i++) pre[i + 1] = max(pre[i], down[kids[i]] + 1);
for (int i = k - 1; i >= 0; i--) suf[i] = max(suf[i + 1], down[kids[i]] + 1);
for (int i = 0; i < k; i++)
    up[kids[i]] = 1 + max({up_p, pre[i], suf[i + 1]});
```

- O(deg) per vertex, O(n) total
- Works for any associative merge (max, gcd, matrix product, ...)
- CSES 1132 *Tree Distances I*: answer(v) = max(down[v], up[v])

Implementation notes for big trees

- $n = 2 \cdot 10^5$ path graph: recursive DFS depth $2 \cdot 10^5$ — fine on most judges with 256 MB stack, **risky** elsewhere
- Rerooting and virtual-tree preprocessing: use an **iterative** DFS (preorder array + reverse sweep)
- Sack: recursion depth = tree height; switch to an explicit stack if the judge's stack is small
- LCT: splay with an explicit stack for pushes (as in the code)

Pitfalls

- Virtual tree: forgetting to add the root when the DP needs it; not resetting per-query arrays (O(n) reset per query = TLE)
- Virtual tree: sorting by vertex id instead of `tin`
- Sack: forgetting to reset global "max" state after a wipe
- Rerooting: double-counting the child's own contribution in `up`
- Rerooting with modular arithmetic: "divide out a child" fails when the value is 0 mod p → prefix/suffix
- LCT: see the LCT pitfall slide; test against a naive forest

Practice problems

- Library Checker *Dynamic Tree Vertex Add Path Sum* (LCT)
- SPOJ DYNACON1 *Dynamic Tree Connectivity*; Luogu P3690 (LCT template)
- Codeforces 600E *Lomsat gelral*, 570D *Tree Requests*, 208E *Blood Cousins*, 741D (sack)
- Codeforces 613D *Kingdom and its Cities*, Luogu P2495 消耗战, Codeforces 1320E (virtual tree)
- CSES 1132, 1133 *Tree Distances I/II*; Codeforces 1092F, 1187E (rerooting)

Summary

- LCT: preferred paths in splay trees; access is everything; O(log n) amortised
- DSU on tree: keep the heavy child's data, re-add light subtrees: O(n log n)
- Virtual tree: marked vertices + adjacent LCAs in DFS order, size < 2k
- Rerooting: down pass + up pass; prefix/suffix when merge is not invertible
- All four were stress-tested against brute force; keep those tests with your notebook